

REFERENCE NOTES

Macros & Metaprogramming

quote · unquote · defmacro · DSLs

four worked examples: log, defgetter, my_unless, HTML

Elixir Training Programme

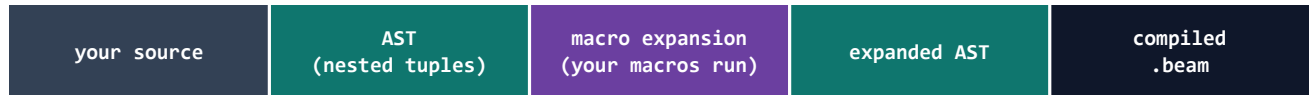
Elixir · metaprogramming, macros and DSLs

Reference notes · Instructor-led · Elixir 1.20.4 on Erlang/OTP 29

METAPROGRAMMING, MACROS & DSLS · REFERENCE NOTES

Code that writes code

Metaprogramming means writing code that writes (or transforms) other code. In Elixir this happens almost exclusively at **compile time**, on the Abstract Syntax Tree (AST). These notes cover the three core tools — **quote**, **unquote** and **defmacro** — and then work through four examples, from a one-line logging macro to a small HTML DSL.



everything before the last box happens while the compiler runs

Contents

#	Topic	What it covers
1	Core concepts	Metaprogramming, the AST, macros, quote / unquote, DSLs
2	Logging with location	A first macro: log/1 prints the caller's file and line
3	Generating functions	defgetter/1 writes a def for you
4	A tiny control-flow DSL	my_unless/2 — your own unless
5	A mini HTML builder	tag/2 and tag/3 — nested markup as Elixir code
6	Key takeaways	What runs when — and when not to write a macro

HOW TO READ Every block marked **REAL OUTPUT** or **REAL EXPANSION** was produced by running the code exactly as listed on Elixir 1.20.4 / Erlang OTP 29. Expansions come from **Macro.expand_once/2** + **Macro.to_string/1**.

1

Core concepts

Metaprogramming, the AST, macros, quote / unquote and DSLs

Metaprogramming means writing code that writes (or transforms) other code. In Elixir this happens almost exclusively at **compile time** via the Abstract Syntax Tree (AST).

Everything you write in Elixir is first turned into an **AST** — a nested tuple structure. Macros receive this AST, can inspect or transform it, and return a new AST that is then compiled.

Macros — the main tool

- Defined with **defmacro**.
- Receive the **AST of their arguments** — not the evaluated values.
- Must **return valid Elixir AST**.
- Are **expanded at compile time** → zero runtime cost for the macro itself.

quote / unquote

- **quote** creates an AST from Elixir code.
- **unquote** injects a value into that AST.
- Together they are the classic **“code as data”** pattern.

QUOTE AND UNQUOTE IN IEX

```
quote(do: sum(1, 2, 3))
# => {:sum, [], [1, 2, 3]}
```

```
quote(do: 1 + 2)
# => {:+, [context: Elixir, imports: [{1, Kernel}, {2, Kernel}]], [1, 2]}
```

```
x = 5
quote(do: 1 + x)           # x stays code: a variable node
# => {:+, [...], [1, {:x, [], Elixir}]}
```

```
quote(do: 1 + unquote(x))  # x's value is injected
# => {:+, [...], [1, 5]}
```

```
Macro.to_string(quote(do: 1 + unquote(x)))
# => "1 + 5"
```

NOTE Every node is a 3-tuple: {**name**, **metadata**, **arguments**}. Without **unquote**, **x** is just the name of a variable in the generated code; with it, the **value** 5 goes into the tree.

DSLs — Domain-Specific Languages

Elixir's syntax is deliberately flexible. By combining **macros + the pipe operator + special forms** you can create highly readable mini-languages for a particular problem domain:

Phoenix
routers

Ecto
queries

ExUnit
tests

Absinthe
GraphQL schemas

2

Simple macro — logging with location

`log/1` prints the caller's file and line in front of the message

LOGGER_MACRO.EXS

```
defmodule LoggerMacro do
  defmacro log(msg) do
    quote do
      IO.puts("#{__ENV__.file}:#{__ENV__.line} #{unquote(msg)}")
    end
  end
end

defmodule Demo do
  require LoggerMacro
  import LoggerMacro

  def run do
    log("Hello from a macro!")
    log("Another message")
  end
end

Demo.run()
```

REAL OUTPUT · ELIXIR LOGGER_MACRO.EXS

```
[../logger_macro.exs:14] Hello from a macro!
[../logger_macro.exs:15] Another message
```

VERIFIED The location is **baked in at compile time**. Lines 14 and 15 are the two `log(...)` calls in the listing above — the notes this is built from said 12 and 13.

WHAT THE COMPILER SEES · REAL EXPANSION OF `LOG("HI")`

```
IO.puts("#{__ENV__.file}:#{__ENV__.line} #{"Hi"})
```

- The macro call is **replaced** by the quoted code. `__ENV__` is quoted too, so it is evaluated where the code lands — in **Demo**, at the caller's line.
- `unquote(msg)` drops the caller's argument AST (here the literal `"Hi"`) into the interpolation.

WATCH OUT `require LoggerMacro` is redundant here: `import` also requires the module. A module with only `import LoggerMacro` compiles and prints the same way (verified).

3

A macro that generates functions

Classic metaprogramming: `defgetter/1` writes a `def` for you

GETTER.EXS

```
defmodule Getter do
  defmacro defgetter(name) do
    quote do
      def unquote(name)(map) do
        Map.get(map, unquote(name))
      end
    end
  end

  defmodule Person do
    require Getter
    import Getter

    defgetter :name
    defgetter :age
  end

  person = %{name: "Alice", age: 30}
  IO.inspect(Person.name(person)) # => "Alice"
  IO.inspect(Person.age(person))  # => 30
end
```

REAL OUTPUT · ELIXIR 1.20.4 / OTP 29

```
"Alice"
30
```

REAL EXPANSION OF DEFGETTER(:NAME)

```
def name(map) do
  Map.get(map, :name)
end
```

THE FUNCTIONS PERSON REALLY HAS

```
Person.__info__(:functions)
# => [age: 1, name: 1]
```

NOTE `unquote(name)` is used twice: once as the **function name** and once as the **map key**. Two lines of `defgetter` became two ordinary public functions at compile time.

4

Tiny DSL — a simple “unless”

Elixir already has **unless**, but here is how you would implement it yourself

CONTROL.EXS

```
defmodule Control do
  defmacro my_unless(condition, do: block) do
    quote do
      if !unquote(condition) do
        unquote(block)
      end
    end
  end
end

defmodule DSLDemo do
  require Control
  import Control

  def run do
    my_unless 1 + 1 == 3 do
      IO.puts("Math still works!")
    end

    my_unless true do
      IO.puts("This will never print")
    end
  end
end

DSLDemo.run()
```

REAL OUTPUT · ELIXIR 1.20.4 / OTP 29

```
Math still works!
```

REAL EXPANSION OF THE FIRST MY_UNLESS

```
if !(1 + 1 == 3) do
  IO.puts("Math still works!")
end
```

- The **do ... end** block reaches the macro as a keyword list **[do: block]**, so the head **my_unless(condition, do: block)** pattern-matches it out.
- Because a macro receives **code, not values**, the block is not run before the macro decides — the second block never executes, so nothing is printed for it.

5

Slightly richer DSL — a mini HTML builder

tag/2 and **tag/3** turn nested Elixir blocks into markup

HTML.EXS

```
defmodule HTML do
  defmacro tag(name, do: block) do
    quote do
      "<#{unquote(name)}>" <> unquote(block) <> "</#{unquote(name)}>"
    end
  end

  defmacro tag(name, attrs, do: block) when is_list(attrs) do
    attr_string =
      attrs
      |> Enum.map(fn {k, v} -> " #{k}=\"#{v}\"" end)
      |> Enum.join()

    quote do
      "<#{unquote(name)}#{unquote(attr_string)}>" <>
        unquote(block) <>
        "</#{unquote(name)}>"
    end
  end
end

defmodule Page do
  require HTML
  import HTML

  def render do
    tag :div, class: "container" do
      tag :h1 do
        "Hello from a macro DSL"
      end <>
      tag :p do
        "This HTML was generated at compile time."
      end
    end
  end
end

IO.puts(Page.render())
```

REAL OUTPUT · ONE LINE, WRAPPED HERE TO FIT

```
<div class="container"><h1>Hello from a macro DSL</h1><p>This HTML was generated at
compile time.</p></div>
```

What runs when

REAL EXPANSION OF TAG(:DIV, [CLASS: "CONTAINER"], DO: "INNER")

```
"<#{:div}#{" class=\"container\""}>" <> "inner" <> "</#{:div}>"
```

At compile time

- Each **tag** call is replaced by the quoted string-building code.
- **attr_string** is computed **inside the macro**, outside **quote** — so " **class**=\"container\"" is already a finished string in the expansion.

At run time — every call to Page.render/0

- The **#{...}** interpolations run.
- The **<>** concatenations build the final HTML string.

CORRECTION The example's own text says the HTML "was generated at compile time". Strictly, the **code that builds it** is generated at compile time; the **string** is assembled each time **Page.render/0** runs. Only the attribute string is finished during compilation.

6

Key takeaways

What each tool does — and when it runs

Concept	What it does	When it runs
quote	Turns code → AST	Compile time
unquote	Injects a value into an AST	Compile time
defmacro	Defines a macro	Compile time
Macro expansion	Replaces the macro call with the returned AST	Compile time
DSL	Combination of macros + Elixir syntax sugar	Compile time

REMEMBER The generated code itself still runs at **run time** — the HTML builder above is the example: expanded while compiling, executed on every call.

RULE OF THUMB Macros are extremely powerful but should be used **sparingly**. Prefer functions when possible; reach for macros only when you need to **change the language itself** — new syntax, compile-time code generation, domain-specific readability.

Try it yourself

SHELL

```
elixir logger_macro.exs      # or paste any example into iex
```